

 ASCENT FORMATION

Langage Rust — débutant

Support de cours complet

Ascent Formation · Formation présentielle · 3 jours (21 h)
Version 2026

Sommaire

Avant-propos

Chapitre 1 — Introduction à Rust

- 1.1 Historique
- 1.2 Objectifs du langage
- 1.3 Avantages et contreparties
- 1.4 Exemples d'applications utilisant Rust

Chapitre 2 — L'environnement de développement

- 2.1 La chaîne d'outils
- 2.2 Installation avec rustup
- 2.3 Configurer l'éditeur
- 2.4 Cargo, le couteau suisse
- 2.5 Premier programme
- 2.6 Lire les messages du compilateur
- 2.7 Tester avec cargo test

Chapitre 3 — Variables, types de données et déclarations

- 3.1 Déclarer une variable
- 3.2 Immutabilité et mutabilité
- 3.3 Le masquage (*shadowing*)
- 3.4 Les types scalaires
- 3.5 Dépassement d'entier (*overflow*)
- 3.6 Conversions entre types
- 3.7 Les types composés
- 3.8 Les chaînes : `String` et `&str`

Chapitre 4 — Possession et emprunt : l'essentiel

- 4.1 Pile et tas
- 4.2 Les trois règles de la possession
- 4.3 Le déplacement (*move*)
- 4.4 L'emprunt (*borrowing*) : les références
- 4.5 La règle d'or des emprunts
- 4.6 Conseils pratiques pour débiter

Chapitre 5 — Fonctions

- 5.1 Définir et appeler une fonction
- 5.2 Instructions et expressions
- 5.3 Retour anticipé et valeurs multiples
- 5.4 Documenter et tester ses fonctions
- 5.5 Aperçu : les fermetures (*closures*)

Chapitre 6 — Structures de contrôle

- 6.1 `if / else if / else`
- 6.2 `match` : le filtrage par motif
- 6.3 Les boucles
- 6.4 Lire une saisie au clavier
- 6.5 Aperçu : les itérateurs

Chapitre 7 — Gestion des erreurs avec `Option` et `Result`

- 7.1 Deux catégories d'erreurs
- 7.2 `Option<T>` : une valeur peut-être absente
- 7.3 `Result<T, E>` : succès ou erreur
- 7.4 Extraire la valeur : `match`, `if let`, `unwrap`, `expect`...
- 7.5 Propager les erreurs avec l'opérateur `?`
- 7.6 Gérer les erreurs de fichiers
- 7.7 Définir ses propres erreurs
- 7.8 Bonnes pratiques

Chapitre 8 — Créer un premier programme Rust structuré

- 8.1 Les structures (`struct`)
- 8.2 Méthodes et fonctions associées (`impl`)
- 8.3 Les énumérations (`enum`)
- 8.4 Concevoir un programme : séparer la logique et les entrées/sorties
- 8.5 Les modules
- 8.6 Arguments de ligne de commande et code de sortie
- 8.7 Compilation et exécution

Chapitre 9 — Pour aller plus loin

Annexe A — Les erreurs de compilation les plus fréquentes

Annexe B — Glossaire

Avant-propos

Ce support accompagne la formation « **Langage Rust — débutant** » (3 jours, 21 heures, présentiel). Il est conçu pour être relu après la formation : chaque chapitre reprend les notions vues en salle, avec des exemples complets que vous pouvez recopier et exécuter.

Comment l'utiliser

- Les blocs de code sont complets et compilent avec Rust édition 2021 (et suivantes).
- Les encadrés **À retenir** résument l'essentiel ; les encadrés **Piège** signalent les erreurs les plus fréquentes des débutants.
- Les exercices pratiques se trouvent dans le *cahier d'exercices* et dans le dossier de code [rust-debutant-exercices/](#). Chaque exercice est livré avec des tests automatiques : votre objectif est de « faire passer les tests au vert ».

Objectifs pédagogiques de la formation

À l'issue de la formation, vous serez capable de :

1. Comprendre les principes fondamentaux du langage Rust.
2. Configurer un environnement de développement Rust.
3. Manipuler des variables, types de données et déclarations en Rust.
4. Utiliser des fonctions et des structures de contrôle (boucles, conditionnelles).
5. Gérer les erreurs avec les types `Option` et `Result`.
6. Écrire un premier programme complet en Rust.

Plan

Jour	Contenu
1	Introduction à Rust · Environnement de développement · Prise en main de Cargo et du compilateur
2	Variables et types · Possession et emprunt (les bases) · Fonctions et structures de contrôle · TP
3	Gestion des erreurs (<code>Option</code> , <code>Result</code> , <code>?</code>) · Structurer un programme (struct, enum, modules) · TP final

Chapitre 1 — Introduction à Rust

1.1 Historique

- **2006** : Graydon Hoare, développeur chez Mozilla, commence Rust comme projet personnel. Le nom évoque un champignon (la rouille des plantes), réputé robuste.
- **2009** : Mozilla sponsorise officiellement le projet. Rust sert à écrire **Servo**, un moteur de navigateur expérimental dont des composants (moteur CSS « Stylo ») intégreront Firefox.
- **15 mai 2015** : sortie de **Rust 1.0**, avec une promesse de stabilité : un code qui compile avec une version 1.x doit compiler avec les versions suivantes.
- **2021** : création de la **Rust Foundation** (membres fondateurs : AWS, Google, Huawei, Microsoft, Mozilla).
- **2022** : Linux 6.1 accepte Rust comme second langage du noyau, après le C.
- Rust évolue par **éditions** (2015, 2018, 2021, 2024) qui permettent des évolutions de syntaxe sans casser le code existant : chaque projet déclare son édition dans `Cargo.toml`, et des crates d'éditions différentes cohabitent sans problème.
- Une nouvelle version stable du compilateur sort **toutes les six semaines**.

1.2 Objectifs du langage

Rust cherche à réconcilier trois qualités souvent opposées :

Objectif	Ce que cela signifie	Comment Rust l'obtient
Performance	Aussi rapide que C/C++	Compilation native (LLVM), pas de ramasse-miettes (<i>garbage collector</i>), abstractions « à coût nul »
Sûreté mémoire	Pas de pointeur invalide, de double libération, de dépassement de tampon, de <i>data race</i>	Système de possession (<i>ownership</i>) et d' emprunt vérifié à la compilation
Productivité	Outils modernes, messages d'erreur clairs	Cargo, rustfmt, clippy, rust-analyzer, documentation intégrée

Le point clé : **la plupart des bugs mémoire sont détectés à la compilation**, pas en production. Des études de Microsoft et de Google ont estimé qu'environ 70 % des failles de sécurité graves de leurs produits provenaient d'erreurs de gestion mémoire — précisément la catégorie de bugs que Rust élimine en grande partie. En France, l'**ANSSI** publie un guide de règles de programmation pour développer des applications sécurisées en Rust, et les agences de cybersécurité américaines (NSA, CISA) recommandent les langages « memory-safe », dont Rust.

1.3 Avantages et contreparties

Avantages

- Sûreté mémoire et concurrence sans *data race*, garanties par le compilateur.
- Performances prévisibles (pas de pause de ramasse-miettes).
- Système de types expressif : `enum` avec données, `match` exhaustif, `Option` au lieu de `null`, `Result` au lieu des exceptions.
- Outillage unifié : un seul outil (Cargo) pour compiler, tester, documenter, gérer les dépendances.
- Messages d'erreur du compilateur parmi les plus pédagogiques de l'industrie.

Contreparties

- Courbe d'apprentissage réelle : la possession et l'emprunt demandent un changement d'habitudes.
- Temps de compilation plus longs que Go ou C.
- Écosystème plus jeune que Java ou Python dans certains domaines.

À retenir — Au début, on a l'impression de « se battre contre le compilateur ». En réalité, il signale des bugs que d'autres langages laisseraient passer jusqu'à l'exécution.

1.4 Exemples d'applications utilisant Rust

Domaine	Exemples
Systèmes d'exploitation	Pilotes du noyau Linux, composants de Windows, Android (Bluetooth, etc.)
Cloud et infrastructure	AWS Firecracker (micro-VM de Lambda/Fargate), Cloudflare (proxy Pingora)
Applications	Discord (services à forte charge), Dropbox (moteur de synchronisation), 1Password
Navigateurs	Firefox (Stylo, composants multimédias)
Outils de développement	ripgrep , fd , bat , uv et Ruff (Python), Deno , Zed (éditeur)
Embarqué, WebAssembly, blockchain	Microcontrôleurs, modules Wasm côté navigateur, Solana, Polkadot

Rust est régulièrement élu **langage le plus « admiré »** par les développeurs dans l'enquête annuelle de Stack Overflow.

Chapitre 2 — L'environnement de développement

2.1 La chaîne d'outils

Outil	Rôle
<code>rustup</code>	Installe et met à jour Rust ; gère plusieurs versions (<i>toolchains</i>) : <code>stable</code> , <code>beta</code> , <code>nightly</code>
<code>rustc</code>	Le compilateur (rarement appelé directement)
<code>cargo</code>	Gestionnaire de projets et de dépendances : compile, exécute, teste, documente
<code>rustfmt</code>	Formateur de code (<code>cargo fmt</code>)
<code>clippy</code>	Analyseur statique (<i>linter</i>) qui suggère du code plus idiomatique (<code>cargo clippy</code>)
<code>rust-analyzer</code>	Serveur de langage : complétion, erreurs en direct, types affichés dans l'éditeur

2.2 Installation avec rustup

Linux et macOS

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
# puis, dans un nouveau terminal (ou après `source "$HOME/.cargo/env"`):
rustc --version
cargo --version
```

- macOS : installer au préalable les outils en ligne de commande : `xcode-select --install`.
- Linux (Debian/Ubuntu) : installer un éditeur de liens C : `sudo apt install build-essential`.

Windows

1. Télécharger et lancer `rustup-init.exe` depuis <https://rustup.rs>.
2. Rust utilise l'éditeur de liens de Microsoft : l'installateur propose d'installer les **Visual Studio Build Tools** (charge de travail « Développement Desktop en C++ »). Acceptez.
3. Fermer puis rouvrir le terminal, et vérifier : `rustc --version`.

Commandes utiles de rustup

```
rustup update          # met à jour vers la dernière version stable
rustup show            # toolchain active et composants
rustup component add clippy rustfmt
rustup doc --book      # le livre officiel, hors ligne !
rustup doc --std       # la documentation de la bibliothèque standard, hors ligne
```

Piège — Sous Windows, l'erreur `linker 'link.exe' not found` signifie que les Build Tools de Visual Studio ne sont pas installés.

Piège — Derrière un proxy d'entreprise, définir `HTTPS_PROXY` (et `HTTP_PROXY`) avant d'installer et avant le premier `cargo build` qui télécharge des dépendances.

2.3 Configurer l'éditeur

Visual Studio Code (recommandé pour la formation)

- Extension **rust-analyzer** (éditeur : *The Rust Programming Language*). N'installez pas l'ancienne extension nommée simplement « Rust », obsolète.
- Extension **CodeLLDB** pour le débogage pas à pas.
- Optionnel : **Even Better TOML** (coloration de `Cargo.toml`), **Error Lens** (erreurs affichées en ligne).
- Réglage conseillé (`settings.json`) : formater à l'enregistrement et utiliser clippy pour l'analyse.

```
{
  "editor.formatOnSave": true,
  "rust-analyzer.check.command": "clippy"
}
```


JetBrains — Le plugin « IntelliJ Rust » a été remplacé par l'IDE dédié **RustRover**, gratuit pour un usage non commercial. Les utilisateurs d'IntelliJ IDEA Ultimate peuvent installer le plugin Rust officiel de JetBrains.

Sans installation — Le **Rust Playground** (<https://play.rust-lang.org>) permet de tester un extrait de code dans le navigateur.

2.4 Cargo, le couteau suisse

```
cargo new bonjour      # crée un projet binaire (dossier bonjour/)
cargo new --lib utils  # crée une bibliothèque
cd bonjour
cargo check            # vérifie que le code compile, SANS produire d'exécutable (rapide)
cargo build            # compile en mode debug -> target/debug/bonjour
cargo run              # compile si besoin, puis exécute
cargo run -- arg1 arg2 # passe des arguments au programme (après --)
cargo build --release  # compile optimisé -> target/release/bonjour
cargo test             # compile et lance les tests
cargo fmt              # formate le code
cargo clippy           # conseils de qualité
cargo doc --open       # génère et ouvre la documentation du projet ET des dépendances
cargo add rand          # ajoute une dépendance (crates.io) dans Cargo.toml
```

Anatomie d'un projet

```
bonjour/
├── Cargo.toml    # manifeste : nom, version, édition, dépendances
├── Cargo.lock    # versions exactes des dépendances (généré, à versionner pour un binaire)
├── src/
│   ├── main.rs  # point d'entrée d'un binaire (fn main)
│   └── lib.rs    # (optionnel) point d'entrée d'une bibliothèque
└── target/      # artefacts de compilation (à ne pas versionner)
```

```
[package]
name = "bonjour"
version = "0.1.0"
edition = "2021"

[dependencies]
rand = "0.8"
```

Debug ou release ? Le mode *debug* compile vite, garde les informations de débogage et vérifie les dépassements d'entiers. Le mode *release* optimise (souvent 10 à 100 fois plus rapide à l'exécution) : c'est lui qu'on livre et qu'on utilise pour mesurer des performances.

Écosystème

- **crates.io** : le registre des bibliothèques (*crates*).
- **docs.rs** : la documentation de toutes les crates publiées, générée automatiquement.
- **lib.rs** : un index alternatif pratique pour découvrir des crates par catégorie.

2.5 Premier programme

```
fn main() {
    println!("Bonjour, le monde !");
}
```

- `fn main()` est le point d'entrée.
- `println!` est une **macro** (le `!` l'indique) : elle vérifie le format **à la compilation**.
- Chaque instruction se termine par `;`.
- L'indentation est de 4 espaces (appliquée par `cargo fmt`).

Formater l'affichage

```
fn main() {
    let nom = "Ferris";
    let age = 9;
    let pi = 3.14159;
    println!("{nom} a {age} ans");           // variables dans le format
    println!("{}", a {} ans", nom, age);    // arguments positionnels
    println!("pi ≈ {pi:.2}");               // 2 décimales : 3.14
    println!("{: >8}", nom);                 // aligné à droite sur 8 : [ Ferris]
    println!("{: <8}", nom);                 // aligné à gauche
    println!("{:?}", (1, "deux", 3.0));     // format de débogage : (1, "deux", 3.0)
    println!("{:#?}", vec![1, 2, 3]);       // débogage « joli », multi-ligne
    eprintln!("ceci part sur la sortie d'erreur");
    let texte = format!("{nom}-{age}");     // comme println!, mais renvoie une String
    println!("{}", texte);
}
```

Commentaires

```
// commentaire de ligne
/* commentaire
   de bloc */

/// Commentaire de documentation de l'élément qui suit (apparaît dans cargo doc).
/// Il accepte le Markdown et les exemples de code (testés par cargo test !).
fn aire(cote: f64) -> f64 {
    cote * cote
}
```

2.6 Lire les messages du compilateur

Un message d'erreur Rust contient :

```
error[E0384]: cannot assign twice to immutable variable `x`
--> src/main.rs:3:5
  |
2 |     let x = 5;
  |         - first assignment to `x`
3 |     x = 6;
  |     ^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
2 |     let mut x = 5;
  |         +++
```

1. Le **code d'erreur** (E0384) : `rustc --explain E0384` affiche une explication détaillée avec exemples.
2. La **position** exacte (fichier:ligne:colonne).
3. Le **contexte** : les lignes concernées, soulignées.
4. Souvent une **aide** (`help:`) qui propose la correction.

À retenir — Méthode : corriger **la première erreur**, recompiler, recommencer. Les erreurs suivantes sont souvent des conséquences de la première. Le compilateur procède aussi par étapes : certaines erreurs (d'emprunt par exemple) n'apparaissent qu'une fois les erreurs de types corrigées.

2.7 Tester avec cargo test

Les tests unitaires s'écrivent dans le même fichier que le code, dans un module annoté `#[cfg(test)]` :

```
pub fn doubler(x: i32) -> i32 {
    x * 2
}

#[cfg(test)]
mod tests {
    use super::*;           // importe les éléments du module parent

    #[test]
    fn double_positif() {
        assert_eq!(doubler(21), 42);
    }

    #[test]
    fn double_negatif() {
        assert!(doubler(-1) < 0);
    }
}
```

- `assert_eq!(a, b)` et `assert_ne!(a, b)` affichent les deux valeurs en cas d'échec.

- `cargo test nom` ne lance que les tests dont le nom contient `nom`.
- `cargo test -p paquet` cible un paquet précis d'un *workspace*.

Chapitre 3 — Variables, types de données et déclarations

3.1 Déclarer une variable

```
let x = 5;           // type inféré : i32
let y: f64 = 2.5;    // type annoté explicitement
let (a, b) = (1, 2); // déstructuration d'un tuple
let z;              // déclaration sans initialisation...
z = 10;             // ...initialisée plus tard (une seule fois)
```

Rust **infère** les types dans la grande majorité des cas. On annoté quand le compilateur ne peut pas deviner (par exemple avec `parse`) ou pour documenter l'intention.

Une variable doit être **initialisée avant d'être lue** : le compilateur le vérifie (erreur [E0381](#)).

3.2 Immutabilité et mutabilité

Par défaut, **les variables sont immuables** :

```
let compteur = 0;
compteur += 1; // ✗ erreur E0384 : cannot assign twice to immutable variable

let mut total = 0; // ✓ `mut` autorise la modification
total += 1;
```

Pourquoi ce choix ? Un code où les valeurs ne changent pas par surprise est plus facile à lire, à tester et à paralléliser. Le mot-clé `mut` rend chaque modification **explicite et visible**.

Constantes et statiques

```
const TVA: f64 = 0.20; // type obligatoire, valeur connue à la compilation
const SECONDES_PAR_JOUR: u32 = 24 * 60 * 60;
static MESSAGE: &str = "Bienvenue"; // emplacement mémoire unique pour tout le programme
```

Convention : `SCREAMING_SNAKE_CASE` pour les constantes, `snake_case` pour variables et fonctions, `PascalCase` pour les types.

3.3 Le masquage (*shadowing*)

On peut redéclarer une variable du même nom avec `let`. La nouvelle **masque** l'ancienne — et peut même changer de type :

```
let saisie = " 42 "; // &str
let saisie = saisie.trim(); // &str sans espaces
let saisie: i32 = saisie.parse().unwrap(); // i32 !
let saisie = saisie * 2; // i32 : 84
```

	let mut	Masquage (let à nouveau)
Change la valeur	Oui	Oui (nouvelle variable)
Peut changer de type	Non	Oui
Usage typique	Compteur, accumulateur	Transformations successives d'une donnée

3.4 Les types scalaires

Entiers

Taille	Signé	Non signé
8 bits	i8 (−128 à 127)	u8 (0 à 255)
16 bits	i16	u16
32 bits	i32 (par défaut)	u32
64 bits	i64	u64
128 bits	i128	u128
taille d'un pointeur	isize	usize (indices, longueurs)

```
let million = 1_000_000; // les _ améliorent la lisibilité
let hexa = 0xff; // 255
let binaire = 0b1010; // 10
let octet = b'A'; // u8 : 65
let petit = 42u8; // suffixe de type
```

Flottants : `f64` (par défaut, double précision) et `f32`.

```
let x = 2.0; // f64
let y: f32 = 3.0;
let division_entiere = 7 / 2; // 3 (entiers : division entière)
let division = 7.0 / 2.0; // 3.5
let reste = 7 % 2; // 1
```

Piège — `9 / 5` vaut `1` (division entière). Pour une conversion °C → °F, écrire `9.0 / 5.0`.

Booléens : `bool`, valeurs `true` / `false`, opérateurs `&&`, `||`, `!`.

Caractères : `char`, entre apostrophes simples, **4 octets**, représente un caractère Unicode : `'a'`, `'é'`, `' '`.

```
let c = 'é';
println!("{}", c.is_alphabetic()); // true
println!("{}", 'A'.to_ascii_lowercase()); // a
```

3.5 Dépassement d'entier (overflow)

```
let x: u8 = 255;
let y = x + 1; // mode debug : panique « attempt to add with overflow »
// mode release : « boucle » silencieusement à 0
```

Pour un comportement explicite, la bibliothèque standard propose des méthodes :

Méthode	250u8.xxx(10)	Intérêt
<code>checked_add</code>	<code>None</code>	Détecter le dépassement (renvoie une <code>Option</code>)
<code>wrapping_add</code>	<code>4</code>	Arithmétique modulaire voulue (hachage...)
<code>saturating_add</code>	<code>255</code>	Plafonner (volume, jauge...)
<code>overflowing_add</code>	<code>(4, true)</code>	Valeur + indicateur

3.6 Conversions entre types

Rust ne fait **aucune conversion implicite** entre types numériques :

```
let entier: i32 = 10;
let flottant: f64 = 2.5;
// let total = entier * flottant; // ✗ E0277 : cannot multiply `i32` by `f64`
let total = entier as f64 * flottant; // ✓ 25.0

let grand: i32 = 300;
let petit = grand as u8; // 44 : `as` TRONQUE sans prévenir
let sur: u8 = u8::try_from(grand).unwrap_or(u8::MAX); // conversion vérifiée : 255
let large: i64 = i64::from(entier); // conversion sans perte : From
```

Du texte vers un nombre : `parse`, qui peut échouer (voir chapitre 7).

```
let n: i32 = "42".parse().unwrap();
let m = "3.5".parse::<f64>().unwrap(); // syntaxe « turbofish » ::<T>
```

3.7 Les types composés

Tuples : regroupent des valeurs de types différents, taille fixe.

```
let personne: (&str, u32, bool) = ("Ada", 36, true);
let (nom, age, _) = personne; // déstructuration ; _ ignore une valeur
println!("{}", a {} ans", personne.0, personne.1);
let unite = (); // le tuple vide « unit » : « pas de valeur »
```

Tableaux : éléments de **même type**, **taille fixe connue à la compilation**, stockés sur la pile.

```
let jours = ["lun", "mar", "mer", "jeu", "ven"]; // [&str; 5]
let zeros = [0; 10]; // dix zéros
println!("{}", jours, le premier est {}", jours.len(), jours[0]);
// jours[10]; // accès hors limites : panique à l'exécution (jamais de lecture mémoire invalide)
```

Vecteurs (`Vec<T>`) : comme un tableau, mais **redimensionnable** (stocké sur le tas).

```
let mut notes: Vec<f64> = Vec::new();
notes.push(12.5);
notes.push(15.0);
let autres = vec![8, 13, 20]; // macro vec!
println!("{}", notes, dernière : {:?}", notes.len(), notes.last());
for n in &notes {
    println!("{}", n);
}
```

3.8 Les chaînes : `String` et `&str`

	<code>String</code>	<code>&str</code>
Nature	Chaîne possédée , modifiable, sur le tas	Vue empruntée sur une chaîne (tranche)
Création	<code>String::from("x")</code> , <code>"x".to_string()</code> , <code>format!</code>	Littéral <code>"x"</code> , <code>&ma_string</code> , <code>&s[0..3]</code>
Usage en paramètre	Quand la fonction doit garder la chaîne	Par défaut pour lire une chaîne

```
let mut s = String::from("Bonjour");
s.push_str(", monde"); // ajoute un &str
s.push('!'); // ajoute un char
let vue: &str = &s; // emprunt : &String se convertit automatiquement en &str
println!("{}", ({ octets, { caractères } }, s, s.len(), s.chars().count());
```

Les chaînes Rust sont encodées en **UTF-8** : `len()` compte des **octets**, pas des caractères (`"é".len()` vaut 2). Pour les caractères : `chars()`.

Méthodes courantes : `trim()`, `to_uppercase()`, `to_lowercase()`, `contains("x")`, `starts_with`, `replace(a, b)`, `split(',')`, `split_whitespace()`, `lines()`, `is_empty()`, `parse::<T>()`.

Chapitre 4 — Possession et emprunt : l'essentiel

C'est la notion originale de Rust. Elle permet la sûreté mémoire sans ramasse-miettes. On en présente ici les bases nécessaires pour écrire des programmes simples.

4.1 Pile et tas

- La **pile** (*stack*) stocke les données de taille connue et fixe (entiers, `bool`, `char`, tableaux, tuples de ces types). Très rapide ; libérée automatiquement à la fin de la fonction.
- Le **tas** (*heap*) stocke les données de taille variable (`String`, `Vec`). Il faut libérer cette mémoire : en C, on appelle `free` (source de bugs) ; en Java, un ramasse-miettes s'en charge ; **en Rust, le propriétaire s'en charge automatiquement quand il sort de portée.**

4.2 Les trois règles de la possession

- Chaque valeur a **un propriétaire** (une variable).
- Il ne peut y avoir **qu'un seul propriétaire à la fois**.
- Quand le propriétaire **sort de sa portée**, la valeur est libérée (*drop*).

```
{
  let s = String::from("bonjour"); // s possède la chaîne
  println!("{s}");
}                                     // fin de portée : la mémoire est libérée ici
```

4.3 Le déplacement (*move*)

```
let a = String::from("Ferris");
let b = a; // la possession PASSE de a à b (move)
// println!("{a}"); // ✗ E0382 : borrow of moved value: `a`
println!("{b}"); // ✓
```

Pourquoi ? Si `a` et `b` possédaient tous deux la chaîne, elle serait libérée deux fois (bug de « double free »). Rust l'interdit à la compilation.

Même chose lors d'un appel de fonction :

```
fn afficher(s: String) { // la fonction PREND POSSESSION de s
  println!("{s}");
} // s est libérée ici

let nom = String::from("Ada");
afficher(nom);
// println!("{nom}"); // ✗ nom a été déplacé dans la fonction
```

Les types `Copy` : les types simples stockés sur la pile (entiers, flottants, `bool`, `char`, et tuples/tableaux de ces types) sont **copiés** au lieu d'être déplacés :

```
let x = 5;
let y = x; // copie
println!("{x} {y}"); // ✓ les deux sont utilisables
```

Cloner : pour obtenir une vraie copie d'une `String` ou d'un `Vec`, on appelle `.clone()` explicitement (l'opération est potentiellement coûteuse, d'où son caractère explicite).

4.4 L'emprunt (*borrowing*) : les références

Plutôt que de donner la possession, on **prête** la valeur avec `&` :

```
fn longueur(s: &String) -> usize { // emprunte, ne possède pas
  s.len()
} // rien n'est libéré : on n'était pas propriétaire

let nom = String::from("Ada");
let n = longueur(&nom); // on prête
println!("{nom} fait {n} octets"); // ✓ nom est toujours utilisable
```

Pour **modifier** une valeur empruntée : référence mutable `&mut`.

```
fn ajouter_point(s: &mut String) {
    s.push('.');
}

let mut phrase = String::from("Fin");
ajouter_point(&mut phrase);
println!("{phrase}");           // Fin.

fn incrementer(n: &mut i32) {
    *n += 1;                     // * « déréférence » : on agit sur la valeur pointée
}
```

4.5 La règle d’or des emprunts

À un instant donné, on peut avoir **soit plusieurs références immuables (&), soit une seule référence mutable (&mut)** — jamais les deux. Et une référence ne doit jamais survivre à la valeur qu’elle désigne.

```
let mut v = vec![1, 2, 3];
let premier = &v[0];           // emprunt immuable
// v.push(4);                  // ✗ E0502 : emprunt mutable pendant un emprunt immuable
println!("{premier}");          // (push pourrait réallouer v : premier pointerait dans le vide)
v.push(4);                      // ✔ ici l'emprunt `premier` n'est plus utilisé
```

Cette règle élimine, à la **compilation**, les pointeurs invalides et les accès concurrents non synchronisés (*data races*).

4.6 Conseils pratiques pour débiter

Besoin de la fonction	Type du paramètre
Lire une chaîne	<code>&str</code>
Lire une liste	<code>&[T]</code> (tranche : accepte <code>&Vec<T></code> et <code>&[T; N]</code>)
Modifier la donnée de l’appelant	<code>&mut T</code>
Conserver la donnée (la stocker dans une struct...)	<code>T</code> (possession)
Types simples (<code>i32</code> , <code>f64</code> , <code>bool</code> , <code>char</code>)	Par valeur, directement

À retenir — Emprunter par défaut, posséder quand c’est nécessaire, cloner en dernier recours (et consciemment).

Chapitre 5 — Fonctions

5.1 Définir et appeler une fonction

```
fn aire_rectangle(largeur: f64, hauteur: f64) -> f64 {
    largeur * hauteur
}

fn saluer(nom: &str) {           // pas de -> : renvoie () (« unit »)
    println!("Bonjour {nom}");
}

fn main() {
    let a = aire_rectangle(3.0, 4.5);
    saluer("Ada");
    println!("{a}");
}
```

- Le **type de chaque paramètre est obligatoire** (pas d'inférence dans les signatures : elles servent de contrat).
- L'ordre de déclaration des fonctions dans le fichier n'a pas d'importance.
- Pas de surcharge, pas de paramètres par défaut, pas d'arguments nommés : on utilise des noms de fonctions distincts ou des structures.

5.2 Instructions et expressions

C'est une notion **fondamentale** en Rust :

- une **expression** produit une valeur : `5`, `a + b`, un appel de fonction, un bloc `{ }`, un `if`, un `match` ;
- une **instruction** effectue une action et ne produit pas de valeur : `let x = 5;`, une expression suivie de `;`.

La dernière expression d'un bloc, sans point-virgule, est la valeur du bloc — et donc la valeur de retour de la fonction :

```
fn carre(x: i32) -> i32 {
    x * x           // ✓ pas de ; : c'est la valeur renvoyée
}

fn carre_faux(x: i32) -> i32 {
    x * x;          // ✗ E0308 : mismatched types, expected `i32`, found `()`
                    // (help: remove this semicolon)
}

let y = {
    let a = 3;
    a * 2           // le bloc vaut 6
};
```

5.3 Retour anticipé et valeurs multiples

```
fn valeur_absolue(x: i32) -> i32 {
    if x < 0 {
        return -x;    // return : sortie anticipée
    }
    x
}

fn min_max(valeurs: &[i32]) -> (i32, i32) {    // plusieurs valeurs : un tuple
    let mut min = valeurs[0];
    let mut max = valeurs[0];
    for &v in valeurs {
        if v < min { min = v; }
        if v > max { max = v; }
    }
    (min, max)
}

let (petit, grand) = min_max(&[4, 8, 1, 9]);
```

5.4 Documenter et tester ses fonctions

```

/// Calcule la moyenne de deux nombres.
///
/// # Exemple
/// ```
/// assert_eq!(mon_crate::moyenne2(10.0, 20.0), 15.0);
/// ```
pub fn moyenne2(a: f64, b: f64) -> f64 {
    (a + b) / 2.0
}

```

L'exemple dans la documentation est **exécuté par cargo test** (*doc-test*) : la documentation ne peut pas devenir fausse sans que les tests échouent.

5.5 Aperçu : les fermetures (closures)

Une fermeture est une fonction anonyme, souvent passée à une méthode d'itérateur :

```

let double = |x: i32| x * 2;
println!("{}", double(21)); // 42
let notes = vec![12, 7, 15];
let admis = notes.iter().filter(|&n| n >= 10).count(); // 2
let existe = notes.iter().any(|&n| n > 14); // true

```

Nous les utiliserons ponctuellement (`any`, `position`, `map`, `sort_by`) ; leur étude détaillée dépasse le cadre d'un cours débutant.

Chapitre 6 — Structures de contrôle

6.1 `if / else if / else`

```
let temperature = 23;
if temperature > 30 {
  println!("canicule");
} else if temperature > 20 {
  println!("agréable");
} else {
  println!("frais");
}
```

- La condition **doit être un bool** : `if 1 { }` est refusé (pas de conversion implicite comme en C ou JavaScript).
- Pas de parenthèses autour de la condition ; les accolades sont obligatoires.

if est une expression : il remplace l'opérateur ternaire.

```
let statut = if age >= 18 { "majeur" } else { "mineur" };
// les deux branches doivent avoir LE MÊME TYPE (sinon E0308)
```

6.2 `match` : le filtrage par motif

`match` compare une valeur à une série de **motifs** et exécute le bras du premier motif qui correspond.

```
let note = 14;
let appreciation = match note {
  20 => "parfait",
  16..=19 => "très bien",           // plage inclusive
  10..=15 => "correct",
  0..=9 => "insuffisant",
  _ => "note invalide",           // _ : tout le reste
};
```

Règle essentielle : un `match` doit être exhaustif. Tous les cas possibles doivent être couverts, sinon le code ne compile pas (E0004). C'est une protection précieuse : si l'on ajoute un cas à un `enum`, le compilateur signale tous les `match` à mettre à jour.

Motifs utiles

```
match jour {
  6 | 7 => println!("week-end"),           // plusieurs valeurs : |
  1..=5 => println!("semaine"),
  _ => println!("inconnu"),
}

match (x % 3, x % 5) {                     // match sur un tuple
  (0, 0) => println!("FizzBuzz"),
  (0, _) => println!("Fizz"),
  (_, 0) => println!("Buzz"),
  _ => println!("{x}"),
}

match n {
  v if v < 0 => println!("négatif"),        // garde : condition supplémentaire
  0 => println!("zéro"),
  v => println!("positif : {v}"),          // liaison : v capture la valeur
}

match commande.as_str() {                 // match sur une String : passer par &str
  "q" | "quitter" => {},
  autre => println!("inconnu : {autre}"),
}
```

if let : raccourci quand un seul motif nous intéresse.

```
let texte = "42";
if let Ok(n) = texte.parse:<i32>() {
  println!("nombre : {n}");
}
```

matches! : teste si une valeur correspond à un motif (renvoie un `bool`).

```
let voyelle = matches!(c, 'a' | 'e' | 'i' | 'o' | 'u' | 'y');
```

6.3 Les boucles

loop : boucle infinie, dont on sort avec **break**. Elle peut **renvoyer une valeur** :

```
let mut p = 1;
let puissance = loop {
    if p > 100 {
        break p;           // la boucle vaut p (128)
    }
    p *= 2;
};
```

while : tant qu'une condition est vraie.

```
let mut n = 3;
while n > 0 {
    println!("{n}...");
    n -= 1;
}
println!("Décollage !");
```

for : parcourt un **itérateur** (plage, tableau, vecteur...). C'est la boucle la plus utilisée — et la plus sûre : pas d'indice hors limites.

```
for i in 0..5 { }           // 0, 1, 2, 3, 4   (borne haute exclue)
for i in 1..=5 { }         // 1 à 5         (borne haute incluse)
for i in (1..=3).rev() { } // 3, 2, 1
for i in (0..10).step_by(2) { } // 0, 2, 4, 6, 8

let noms = vec!["Ada", "Grace", "Linus"];
for nom in &noms { }        // emprunt : noms reste utilisable
for (i, nom) in noms.iter().enumerate() { // indice + valeur
    println!("{i} : {nom}");
}
```

Piège — **for nom in noms** (sans **&**) **consomme** le vecteur : il est déplacé dans la boucle et n'est plus utilisable ensuite. Utilisez **&noms** pour le parcourir en lecture, **&mut noms** pour le modifier.

continue passe à l'itération suivante ; **break** sort de la boucle. Les **étiquettes** permettent de sortir d'une boucle externe :

```
'externe: for i in 0..10 {
    for j in 0..10 {
        if i * j == 42 {
            println!("trouvé : {i} × {j}");
            break 'externe;
        }
    }
}
```

6.4 Lire une saisie au clavier

```
use std::io::{self, Write};

fn main() {
    print!("Votre nom : ");
    io::stdout().flush().expect("flush impossible"); // print! ne vide pas le tampon

    let mut saisie = String::new();
    io::stdin()
        .read_line(&mut saisie)           // ajoute la ligne, \n compris
        .expect("lecture impossible");

    let nom = saisie.trim();               // retire espaces et \n
    println!("Bonjour {nom} !");
}
```

read_line renvoie le nombre d'octets lus : **0** signifie la fin de l'entrée (Ctrl+D sous Linux/macOS, Ctrl+Z puis Entrée sous Windows).

6.5 Aperçu : les itérateurs

Les boucles **for** reposent sur les itérateurs, qui offrent de nombreuses méthodes chaînables :

```
let v = vec![3, 8, 1, 6];
let somme: i32 = v.iter().sum(); // 18
let doubles: Vec<i32> = v.iter().map(|x| x * 2).collect(); // [6, 16, 2, 12]
let pairs = v.iter().filter(|&x| x % 2 == 0).count(); // 2
let max = v.iter().max(); // Some(8)
let factorielle: u64 = (1..=10).product(); // 3628800
```

Chapitre 7 — Gestion des erreurs avec `Option` et `Result`

7.1 Deux catégories d'erreurs

Catégorie	Exemple	Mécanisme Rust
Irrécupérable : un bug, un état impossible	Indice hors limites, invariant violé	<code>panic!</code> : le programme s'arrête avec un message
Récupérable : un événement normal et prévisible	Fichier absent, saisie invalide, valeur introuvable	Les types <code>Option<T></code> et <code>Result<T, E></code>

Rust **n'a ni exceptions, ni `null`**. Une fonction qui peut échouer le dit **dans son type de retour**, et le compilateur oblige l'appelant à traiter le cas d'échec.

```
panic!("configuration corrompue : {}", raison); // arrêt immédiat avec message
```

7.2 `Option<T>` : une valeur peut-être absente

```
enum Option<T> {    // défini dans la bibliothèque standard (importé automatiquement)
    Some(T),        // il y a une valeur
    None,           // il n'y en a pas
}
```

Exemples de fonctions de la bibliothèque standard qui renvoient une `Option` : `v.first()`, `v.get(i)`, `v.iter().max()`, `s.chars().next()`, `s.find('x')`, `v.iter().position(...)`.

```
fn premier_paire(valeurs: &[i32]) -> Option<i32> {
    for &v in valeurs {
        if v % 2 == 0 {
            return Some(v);
        }
    }
    None
}

fn main() {
    match premier_paire(&[3, 7, 10, 4]) {
        Some(n) => println!("premier pair : {n}"),
        None => println!("aucun nombre pair"),
    }
}
```

À retenir — On ne peut pas utiliser une `Option<i32>` comme un `i32` : il faut d'abord **extraire** la valeur, donc traiter le cas `None`. Le fameux « bug à un milliard de dollars » (le `null` oublié) devient impossible.

7.3 `Result<T, E>` : succès ou erreur

```
enum Result<T, E> {
    Ok(T),    // succès, avec la valeur
    Err(E),   // échec, avec l'erreur
}
```

```
fn diviser(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err(String::from("division par zéro"))
    } else {
        Ok(a / b)
    }
}

fn main() {
    match diviser(10.0, 0.0) {
        Ok(q) => println!("résultat : {q}"),
        Err(e) => println!("erreur : {e}"),
    }

    match "42a".parse::<i32>() {           // parse renvoie Result<i32, ParseIntError>
        Ok(n) => println!("{n}"),
        Err(e) => println!("saisie invalide : {e}"), // invalid digit found in string
    }
}
```

7.4 Extraire la valeur : `match`, `if let`, `unwrap`, `expect`...

Méthode	Si <code>Some(v)</code> / <code>Ok(v)</code>	Si <code>None</code> / <code>Err(e)</code>	Quand l'utiliser
<code>match</code> / <code>if let</code>	Traite <code>v</code>	Traite l'erreur	Cas général
<code>unwrap()</code>	<code>v</code>	panique	Prototypes, tests, cas « impossibles »
<code>expect("msg")</code>	<code>v</code>	panique avec <code>msg</code>	Comme <code>unwrap</code> , mais documente pourquoi cela ne devrait pas échouer
<code>unwrap_or(default)</code>	<code>v</code>	<code>default</code>	Valeur par défaut simple
<code>unwrap_or_else(\ ...)</code>	<code>v</code>	Calcule un défaut	Défaut coûteux à calculer
<code>unwrap_or_default()</code>	<code>v</code>	<code>0</code> , <code>"</code> , <code>false</code> ...	Défaut « naturel » du type
<code>?</code>	<code>v</code>	Renvoie l'erreur à l'appelant	Dans une fonction qui renvoie elle-même <code>Option</code> / <code>Result</code>

```
let n: i32 = "12".parse().unwrap(); // ok ici : littéral connu
let port: u16 = std::env::var("PORT")
    .ok() // Result -> Option
    .and_then(|p| p.parse().ok())
    .unwrap_or(8080); // défaut si absent ou invalide
let config = std::fs::read_to_string("config.toml")
    .expect("config.toml doit exister à la racine du projet");
```

Piège — Un `unwrap()` en production est une panique qui attend son heure. Règle simple : `unwrap` autorisé dans les tests et les prototypes ; ailleurs, `expect` avec un message qui explique l'hypothèse, ou mieux, un traitement de l'erreur.

Transformer sans extraire

```
let longueur: Option<usize> = Some("Rust").map(|s| s.len()); // Some(4)
let r: Result<i32, String> = "7".parse::<i32>()
    .map_err(|e| format!("nombre invalide : {e}")); // change le type d'erreur
let o: Option<i32> = "7".parse::<i32>().ok(); // Result -> Option
let r2: Result<i32, &str> = Some(3).ok_or("valeur absente"); // Option -> Result
```

7.5 Propager les erreurs avec l'opérateur `?`

Sans `?`, propager une erreur est verbeux :

```
use std::num::ParseIntError;

fn somme_verbeuse(a: &str, b: &str) -> Result<i32, ParseIntError> {
    let x = match a.trim().parse::<i32>() {
        Ok(v) => v,
        Err(e) => return Err(e),
    };
    let y = match b.trim().parse::<i32>() {
        Ok(v) => v,
        Err(e) => return Err(e),
    };
    Ok(x + y)
}
```

Avec `?` : si c'est `Ok(v)`, on continue avec `v` ; si c'est `Err(e)`, la fonction renvoie immédiatement `Err(e)`.

```
fn somme(a: &str, b: &str) -> Result<i32, ParseIntError> {
    let x: i32 = a.trim().parse()?;
    let y: i32 = b.trim().parse()?;
    Ok(x + y)
}
```

Conditions d'utilisation de `?` :

1. La fonction doit renvoyer un `Result` (ou une `Option` si on applique `?` à une `Option`).
2. Le type d'erreur doit être compatible (identique, ou convertible via le trait `From`). Sinon, convertir d'abord avec `map_err` ou `ok_or`.

`?` fonctionne aussi avec `Option` :

```
fn initiale_nom_famille(nom_complet: &str) -> Option<char> {
    let nom = nom_complet.split_whitespace().nth(1)?; // None si un seul mot
    nom.chars().next()
}
```

`main` peut renvoyer un `Result`, ce qui autorise `?` dans `main` :

```
use std::error::Error;
use std::fs;

fn main() -> Result<(), Box<dyn Error>> {
    let contenu = fs::read_to_string("donnees.txt")?;
    let n: i32 = contenu.trim().parse()?; // deux types d'erreur différents :
    println!("valeur lue : {n}"); // Box<dyn Error> accepte les deux
    Ok(())
}
```

7.6 Gérer les erreurs de fichiers

```
use std::fs;
use std::io::{self, ErrorKind};

fn compter_lignes(chemin: &str) -> Result<usize, io::Error> {
    let contenu = fs::read_to_string(chemin)?;
    Ok(contenu.lines().count())
}

fn main() {
    match compter_lignes("notes.txt") {
        Ok(n) => println!("{n} lignes"),
        Err(e) if e.kind() == ErrorKind::NotFound => {
            eprintln!("Fichier introuvable : vérifiez le chemin.");
        }
        Err(e) if e.kind() == ErrorKind::PermissionDenied => {
            eprintln!("Accès refusé.");
        }
        Err(e) => eprintln!("Erreur inattendue : {e}"),
    }
}
```

Écrire dans un fichier : `fs::write("sortie.txt", contenu)?;` (crée ou remplace le fichier).

7.7 Définir ses propres erreurs

Une `String` comme type d'erreur suffit pour débiter, mais un `enum d'erreurs` permet à l'appelant de réagir différemment selon le cas — et le `match` exhaustif garantit qu'aucun cas n'est oublié.


```

use std::fmt;

#[derive(Debug, PartialEq)]
enum ErreurSaisie {
    Vide,
    PasUnNombre(String),
    HorsLimites(i32),
}

impl fmt::Display for ErreurSaisie {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            ErreurSaisie::Vide => write!(f, "saisie vide"),
            ErreurSaisie::PasUnNombre(t) => write!(f, "« {t} » n'est pas un nombre"),
            ErreurSaisie::HorsLimites(n) => write!(f, "{n} n'est pas entre 1 et 10"),
        }
    }
}

fn lire_note(saisie: &str) -> Result<i32, ErreurSaisie> {
    let texte = saisie.trim();
    if texte.is_empty() {
        return Err(ErreurSaisie::Vide);
    }
    let n: i32 = texte
        .parse()
        .map_err(|_| ErreurSaisie::PasUnNombre(texte.to_string()))?;
    if !(1..=10).contains(&n) {
        return Err(ErreurSaisie::HorsLimites(n));
    }
    Ok(n)
}

fn main() {
    for essai in ["7", "", "abc", "42"] {
        match lire_note(essai) {
            Ok(n) => println!("note acceptée : {n}"),
            Err(e) => println!("refusé : {e}"), // utilise Display
        }
    }
}

```

Dans les projets réels, les crates `thiserror` (pour définir des erreurs dans une bibliothèque) et `anyhow` (pour une application) réduisent ce code.

7.8 Bonnes pratiques

1. Une fonction qui peut échouer renvoie `Result` ; une recherche qui peut ne rien trouver renvoie `Option`.
2. Propager avec `?` dans les couches internes ; **traiter** (afficher, réessayer, valeur par défaut) au niveau de l'interface utilisateur (`main`).
3. Des messages d'erreur utiles : **quoi**, **où** (numéro de ligne, nom de fichier), **comment corriger**.
4. Réserver `panic!`, `unwrap` et `expect` aux bugs et aux hypothèses vérifiées.
5. Afficher les erreurs sur `stderr` (`eprintln!`) et quitter avec un code non nul (`std::process::exit(1)`).

Chapitre 8 — Créer un premier programme Rust structuré

8.1 Les structures (`struct`)

Une structure regroupe des données nommées :

```
#[derive(Debug, Clone, PartialEq)] // traits générés automatiquement
struct Contact {
    nom: String,
    email: String,
    age: u32,
}

fn main() {
    let ada = Contact {
        nom: String::from("Ada"),
        email: String::from("ada@exemple.fr"),
        age: 36,
    };
    println!("{}", <{}>, ada.nom, ada.email);
    println!("{}", <{:?}>, ada); // grâce à #[derive(Debug)]
    let mut copie = ada.clone(); // grâce à #[derive(Clone)]
    copie.age += 1; // modification : la variable doit être mut
}
```

derive	Apporte
Debug	Affichage <code>{:?}</code>
Clone	Méthode <code>.clone()</code>
Copy	Copie implicite (seulement si tous les champs sont <code>Copy</code>)
PartialEq	Comparaison <code>==</code> (et <code>assert_eq!</code> dans les tests)
Default	Valeur par défaut <code>Type::default()</code>

8.2 Méthodes et fonctions associées (`impl`)

```
struct Rectangle {
    largeur: f64,
    hauteur: f64,
}

impl Rectangle {
    // fonction associée (pas de self) : sert de constructeur, appelée avec ::
    fn new(largeur: f64, hauteur: f64) -> Rectangle {
        Rectangle { largeur, hauteur } // raccourci quand variable et champ ont le même nom
    }

    fn carre(cote: f64) -> Self { // Self = le type en cours
        Self { largeur: cote, hauteur: cote }
    }

    // méthode en lecture : &self
    fn aire(&self) -> f64 {
        self.largeur * self.hauteur
    }

    // méthode qui modifie : &mut self
    fn agrandir(&mut self, facteur: f64) {
        self.largeur *= facteur;
        self.hauteur *= facteur;
    }
}

fn main() {
    let mut r = Rectangle::new(3.0, 4.0);
    r.agrandir(2.0);
    println!("aire : {}", r.aire()); // 48
    let c = Rectangle::carre(5.0);
    println!("carré : {}", c.aire());
}
```

Afficher avec `{}` : implémenter le trait `Display`.

```
use std::fmt;

impl fmt::Display for Rectangle {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{} x {}", self.largeur, self.hauteur)
    }
}
// println!("{}", r) affiche « 6 x 8 » ; r.to_string() est disponible gratuitement
```

8.3 Les énumérations (enum)

Un `enum` définit un type qui est **l'une de plusieurs variantes**, chacune pouvant porter des données :

```
enum Commande {
    Ajouter { nom: String, email: String }, // variante avec champs nommés
    Supprimer(String),                    // variante avec une valeur
    Lister,                                // variante sans donnée
    Quitter,
}

fn executer(cmd: Commande) {
    match cmd {
        Commande::Ajouter { nom, email } => println!("ajout de {nom} <{email}>"),
        Commande::Supprimer(nom) => println!("suppression de {nom}"),
        Commande::Lister => println!("liste..."),
        Commande::Quitter => println!("au revoir"),
    }
}
```

`Option` et `Result` sont simplement des `enum` génériques de la bibliothèque standard. Combinés à `match`, les `enum` permettent de **rendre les états invalides impossibles à représenter**.

8.4 Concevoir un programme : séparer la logique et les entrées/sorties

Une architecture simple et efficace pour un programme en ligne de commande :

```
mon_programme/
├── Cargo.toml
├── src/
│   ├── lib.rs    # la LOGIQUE : types, calculs, validation – sans println!, sans stdin
│   │             # → facile à tester avec cargo test
│   └── main.rs   # l'INTERFACE : lit les arguments/la saisie, appelle lib.rs, affiche
```

Dans `main.rs`, on utilise la bibliothèque du même paquet par son nom :

```
// src/main.rs (le paquet s'appelle « mon_programme »)
use mon_programme::{Carnet, Contact};

fn main() {
    let mut carnet = Carnet::new();
    // ...
}
```

Démarche de conception

1. Écrire en une phrase ce que fait le programme, puis lister les **cas d'usage**.
2. Identifier les **données** → `struct`, et les **alternatives** (commandes, états, erreurs) → `enum`.
3. Écrire les **signatures** des fonctions de `lib.rs` avec des `todo!()`, puis les **tests**.
4. Implémenter jusqu'à ce que les tests passent.
5. Écrire `main.rs` : boucle de saisie, affichage, traitement des erreurs.
6. `cargo fmt`, `cargo clippy`, `cargo test`, puis `cargo build --release`.

Astuce — La macro `todo!()` compile quel que soit le type attendu et panique à l'exécution : idéale pour poser le squelette d'un programme.

8.5 Les modules

Quand un fichier grossit, on le découpe en **modules** :

```
// src/lib.rs
pub mod contact; // cherche le code dans src/contact.rs
pub mod carnet;  // cherche le code dans src/carnet.rs

pub use carnet::Carnet; // ré-exporte pour simplifier l'import côté utilisateur
```

```
// src/contact.rs
#[derive(Debug, Clone)]
pub struct Contact {
    pub nom: String, // pub : visible hors du module
    pub email: String, // les champs aussi doivent être pub pour être accessibles
}
```

```
// src/carnet.rs
use crate::contact::Contact; // crate:: = racine du paquet

pub struct Carnet {
    contacts: Vec<Contact>, // champ privé : on passe par les méthodes
}
```

- Tout est **privé par défaut** ; `pub` rend public.
- `use` crée un raccourci vers un chemin (`std::collections::HashMap`, `crate::contact::Contact`).
- `self::` désigne le module courant, `super::` le module parent.

8.6 Arguments de ligne de commande et code de sortie

```
use std::env;
use std::process;

fn main() {
    let args: Vec<String> = env::args().collect(); // args[0] = nom du programme
    if args.len() < 2 {
        eprintln!("Usage : {} <fichier>", args[0]);
        process::exit(2); // code de sortie non nul = échec
    }
    let chemin = &args[1];
    println!("Traitement de {chemin}");
}
```

Exécution : `cargo run -- donnees.csv`. Pour des interfaces riches (options, aide automatique), la crate `clap` est la référence.

8.7 Compilation et exécution

```
cargo build # debug : target/debug/mon_programme
cargo build --release # release : target/release/mon_programme(.exe)
./target/release/mon_programme donnees.csv
```

- Le binaire produit est **autonome** : il ne nécessite pas Rust sur la machine cible (seulement le même système d'exploitation et la même architecture).
- `cargo install --path .` installe le programme dans `~/.cargo/bin` pour l'utiliser comme une commande.
- Pour viser une autre plateforme : `rustup target add <cible>` puis `cargo build --target <cible>` (compilation croisée).

Chapitre 9 — Pour aller plus loin

Cette formation pose les fondations. Les étapes suivantes, dans l'ordre conseillé :

1. **Collections** : `HashMap`, `HashSet`, `VecDeque`, `BTreeMap`.
2. **Traits et génériques** : définir des comportements communs (`trait`), écrire du code générique (`fn plus_grand<T: PartialOrd>(...)`).
3. **Durées de vie** (*lifetimes*) : les annotations `'a` qui expliquent au compilateur la relation entre références.
4. **Itérateurs et fermetures** en profondeur.
5. **Pointeurs intelligents** : `Box`, `Rc`, `RefCell`, `Arc`, `Mutex`.
6. **Concurrence** : threads, canaux, `Arc<Mutex<T>>`, puis programmation asynchrone avec `async/await` et `tokio`.
7. **Écosystème** : `serde` (sérialisation JSON/TOML...), `clap` (ligne de commande), `anyhow` / `thiserror` (erreurs), `request` (HTTP), `axum` (web), `sqlx` (bases de données).

Ressources recommandées

Ressource	Description
<i>The Rust Programming Language</i> (« le Book »)	Le livre officiel, gratuit en ligne ; traduction française communautaire : « Le langage de programmation Rust »
<i>Rust by Example</i>	Apprentissage par exemples exécutables
Rustlings	Petits exercices à corriger dans le terminal, guidés par le compilateur
Exercism (piste Rust)	Exercices avec retours de mentors bénévoles
<i>Comprehensive Rust</i> (Google)	Cours complet de plusieurs jours, conçu pour des développeurs expérimentés
docs.rs et <code>rustup doc --std</code>	Documentation de référence
Guide ANSSI	Règles de programmation pour développer des applications sécurisées en Rust

Annexe A — Les erreurs de compilation les plus fréquentes

Code	Message (résumé)	Cause habituelle	Correction
E0384	<i>cannot assign twice to immutable variable</i>	Modification d'une variable non <code>mut</code>	<code>let mut</code>
E0308	<i>mismatched types</i>	Type inattendu ; souvent un <code>;</code> en trop en fin de fonction, ou des branches <code>if</code> de types différents	Retirer le <code>;</code> , harmoniser les types, convertir
E0277	<i>cannot add <code>f64</code> to <code>i32</code> (trait non implémenté)</i>	Opération entre types différents	Conversion explicite <code>as f64</code>
E0382	<i>borrow of moved value</i>	Valeur utilisée après avoir été déplacée	Emprunter (<code>&</code>), cloner, ou réorganiser
E0502	<i>cannot borrow as mutable because it is also borrowed as immutable</i>	Mélange <code>&</code> et <code>&mut</code> simultanés	Terminer l'utilisation de <code>&</code> avant <code>&mut</code>
E0499	<i>cannot borrow as mutable more than once</i>	Deux <code>&mut</code> en même temps	Réduire la portée des emprunts
E0596	<i>cannot borrow as mutable</i>	<code>&mut</code> sur une variable non <code>mut</code>	<code>let mut</code>
E0004	<i>non-exhaustive patterns</i>	<code>match</code> incomplet	Ajouter les cas manquants ou <code>_</code>
E0425	<i>cannot find value in this scope</i>	Faute de frappe, variable hors de portée	Vérifier le nom et la portée
E0599	<i>no method named ... found</i>	Méthode inexistante sur ce type, trait non importé	Vérifier le type (survol dans l'éditeur), <code>use</code> du trait
E0381	<i>used binding isn't initialized</i>	Variable lue avant initialisation	Initialiser sur tous les chemins
E0106	<i>missing lifetime specifier</i>	Fonction renvoyant une référence ambiguë	Renvoyer une valeur possédée (<code>String</code>) pour débiter

Annexe B — Glossaire

Terme	Définition
<i>Borrow</i> (emprunt)	Accès temporaire à une valeur via une référence <code>&</code> ou <code>&mut</code> , sans en prendre possession
<i>Borrow checker</i>	Partie du compilateur qui vérifie les règles d'emprunt
<i>Crate</i>	Unité de compilation : une bibliothèque ou un exécutable ; publiée sur crates.io
<i>Drop</i>	Libération automatique d'une valeur quand son propriétaire sort de portée
Expression	Morceau de code qui produit une valeur
Macro	Code qui génère du code à la compilation ; reconnaissable au <code>!</code> (<code>println!</code> , <code>vec!</code>)
<i>Move</i> (déplacement)	Transfert de la possession d'une valeur d'une variable à une autre
<i>Ownership</i> (possession)	Système de règles qui attribue à chaque valeur un propriétaire unique
<i>Panic</i>	Arrêt du programme sur une erreur irrécupérable
<i>Shadowing</i> (masquage)	Redéclaration d'une variable du même nom avec <code>let</code>
<i>Slice</i> (tranche)	Vue sur une partie contiguë d'une collection : <code>&[T]</code> , <code>&str</code>
<i>Trait</i>	Ensemble de méthodes qu'un type peut implémenter (équivalent proche d'une interface)
<i>Workspace</i>	Ensemble de paquets Cargo partageant un même <code>Cargo.lock</code> et un même dossier <code>target/</code>